

LAB 3: Propositional Logic & Model Checking

Group Challenge

Instructor	Dr. Naila Marir	Duration	1 hour 30 minutes
Semester	Fall 2026	Group size	3–4 students
Submission	One GitHub link per group	Total marks	100
Group No.			
Member name	Student ID	Member name	Student ID
Malaika Tanveer	S24209791	Yumna Sarajuddin	S24109478
Jumaana Musa	S24109373		

1. Objectives

1. Represent facts as propositional logic sentences using Not, And, Or and Implication.
2. Explain how `model_check` decides whether a knowledge base entails a query.
3. Encode a new logic puzzle in Python and solve it with model checking.

2. Tools and Files

1. Python 3 and the package `termcolor` (install with `pip install termcolor`).
2. Lab files (on Blackboard): `logic.py`, `harry.py`, `clue.py`, `puzzle.py`, `mastermind.py`. Put all of them in one folder.
3. A GitHub account for each group.

3. Lab Plan

Step	Activity	Time
0	Setup: run <code>harry.py</code> and <code>clue.py</code>	5 min
1	Task 1 — Understand the code	20 min
2	Task 2 — Change the knowledge	20 min
3	Task 3 — Knights & Knaves challenge	30 min
4	Write the PDF and push to GitHub	15 min

4. Task 1 — Understand the Code (30 marks, 20 min)

1. **harry.py:** Write the truth table for `rain`, `hagrid`, `dumbledore` (8 rows). Mark the rows where the knowledge base is true. Use the table to explain why the program prints `True`.
2. **clue.py:** The program prints `MsScarlet`, `library` and `knife`. Explain in plain English why each one must be true.

5. Task 2 — Change the Knowledge (30 marks, 20 min)

For each change: **(a)** predict the result, **(b)** run the code, **(c)** explain the result.

1. **harry.py:** Delete the last fact `dumbledore` from the knowledge base. What does the program print now? Why?
2. **puzzle.py:** Delete the clue `MinervaGryffindor`. Who can still be placed in a house? Then add **one new clue of your own** that makes the answer unique again.

6. Task 3 — Group Challenge: Knights & Knaves (30 marks, 30 min)

On an island, a **knight** always tells the truth and a **knave** always lies. Every person is one or the other, never both. Create a new file `knights.py` that imports `logic.py` and solves the puzzles below.

Hint — use this pattern for what a person says:

```
Implication(AKnight, statement)      # a knight's statement is true
Implication(AKnave, Not(statement))  # a knave's statement is false
```

1. **Puzzle 0:** A says “I am both a knight and a knave.”
2. **Puzzle 1:** A says “We are both knaves.” B says nothing.
3. **Puzzle 2:** A says “We are the same kind.” B says “We are of different kinds.”

Bonus (+10 marks) — Puzzle 3: A says either “I am a knight.” or “I am a knave.” (you don’t know which). B says “A said ‘I am a knave.’” B also says “C is a knave.” C says “A is a knight.”

7. Submission

Create **one public GitHub repository** named `Lab3_GroupX` and submit **only its link** on Blackboard. The repository must contain:

1. `Lab3_GroupX.pdf` — group number, member names, and for each task: your answer, your code, and a screenshot of the output.
2. All `.py` files — `logic.py`, your changed `harry.py` and `puzzle.py`, and your new `knights.py`.
3. `README.md` — member names and one line on how to run `knights.py`.

Before you submit: open your link in a private (incognito) browser window to check that it works.

8. Grading Rubric

Task	What is checked	Marks
Task 1	Correct truth table and clear explanations	30
Task 2	Predictions, results and explanations; a working new clue	30
Task 3	Puzzles 0–2 encoded correctly with output and explanation	30
Submission	Clear PDF and complete GitHub repository	10
Total		100
Bonus	Puzzle 3 solved and explained	+10

REPORT

Knowledge Base:

$(\neg R \rightarrow H) \wedge (H \vee D) \wedge \neg(H \wedge D) \wedge D$

where:

R = rain

H = hagrid

D = dumbledore

<u>R</u>	<u>H</u>	<u>D</u>	<u>(NOT R) - > H</u>	<u>H OR D</u>	<u>NOT(H AND D)</u>	<u>D</u>	<u>KB</u>
<u>T</u>	<u>T</u>	<u>T</u>	<u>T</u>	<u>T</u>	<u>F</u>	<u>T</u>	<u>F</u>
<u>T</u>	<u>T</u>	<u>F</u>	<u>T</u>	<u>T</u>	<u>T</u>	<u>F</u>	<u>F</u>
<u>T</u>	<u>F</u>	<u>T</u>	<u>T</u>	<u>T</u>	<u>T</u>	<u>T</u>	<u>T</u>
<u>T</u>	<u>F</u>	<u>F</u>	<u>T</u>	<u>F</u>	<u>T</u>	<u>F</u>	<u>F</u>
<u>F</u>	<u>T</u>	<u>T</u>	<u>T</u>	<u>T</u>	<u>F</u>	<u>T</u>	<u>F</u>
<u>F</u>	<u>T</u>	<u>F</u>	<u>T</u>	<u>T</u>	<u>T</u>	<u>F</u>	<u>F</u>
<u>F</u>	<u>F</u>	<u>T</u>	<u>F</u>	<u>T</u>	<u>T</u>	<u>T</u>	<u>F</u>
<u>F</u>	<u>F</u>	<u>F</u>	<u>F</u>	<u>F</u>	<u>T</u>	<u>F</u>	<u>F</u>

Task 1 – Understand The Code

a. The knowledge base is true only when rain = True, hagrid = False, and dumbledore = True. Since every model that satisfies the knowledge base has rain = True, the knowledge base entails rain. Therefore, model_check(knowledge, rain) returns True.

b. Output:

```
MsScarlet: YES
library: YES
knife: YES
```

Result: Ms Scarlet is the suspect, the library is the room, and the knife is the weapon.

Reason: Colonel Mustard, Professor Plum, the kitchen, ballroom, and revolver are eliminated by the knowledge base. The remaining valid solution is Ms Scarlet in the library with the knife.

Task 2 – Change the Knowledge

1. Prediction: False, Result: False,

Explanation: After removing Dumbledore from the knowledge base, rain is no longer guaranteed to be true. There are now valid models where rain is false, so `model_check(knowledge, rain)` returns False.

```
knowledge = And(  
    Implication(Not(rain), hagrid),  
    Or(hagrid, dumbledore),  
    Not(And(hagrid, dumbledore)),  
)
```

False

2. Prediction: I think the program will not give one clear answer, Result: The program does not print one fixed house assignment.

Explanation: After removing the Minerva clue, there is more than one possible arrangement, so the program cannot decide on one unique answer.

New clue: Minerva is in Ravenclaw.

```
knowledge.add(  
    symbol("MinervaRavenclaw")  
)
```

Output: GilderoyGryffindor
PomonaHufflepuff
MinervaRavenclaw
HoraceSlytherin

Explanation: The new clue removes the remaining uncertainty, so only one house arrangement is possible.

Task 3 – Knights & Knaves

Puzzle 0:

A is a knave. His statement cannot be true.

Puzzle 1:

A is a knave and B is a knight.

Puzzle 2:

A is a knave and B is a knight.

Program output:

```
Puzzle 0  
AKnave  
  
Puzzle 1  
AKnave  
BKnight  
  
Puzzle 2  
AKnave  
BKnight
```